

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Q1 - AT 1

Python program that generates a star topology for a given number of hosts. Connected to a central switch.

```
import random
```

```
n = int(input("Enter number of hosts: "))
```

```
switch = "switch-1"
```

```
for i in range(1, n+1):
```

```
    host = "HOST " + str(i)
```

```
    length = random.randint(5, 100)
```

```
    print(host, "→", switch, "→", length, "m")
```

```
print("All hosts connected to", switch).
```

Explanation:

1. creates the required no. of hosts.
2. connects every host to one central switch.
3. Displays the cat6 cable length for each host.

Q2:

write a Python function named validate_segment(length_m) to verify whether a given vlp cable segment is satisfied.

```
def validate_segment(length_m):
```

```
    if 0 <= length_m <= 100:
```

```
        return True, "valid"
```

```

return False, "Invalid! exceeds 100 meters"
for x in [50, 100, 120]:
    print(x, "m:", validate_segment(x))

```

1. checks whether the cable length is within 100 meters.
2. Returns True for valid lengths and false for invalid lengths.
3. Tests three values: 50 m, 100m and 120m.

Q3. Develop a Python program to simulate the operation of the carrier sense multiple Access with Collision Detection.

```

import random
hosts = ["H1", "H2", "H3", "H4"]
with open("csma_log.txt", "w") as f:
    for host in hosts:
        attempt = 1
        while True:
            print(host, "Attempt", attempt)
            f.write(f"{host} Attempt {attempt}\n")
            if random.choice([True, False]):
                print(f"Collision! Back-off\n")

```

attempt + 1.

else:

Print ("Transmission successful")

f.write ("successful transmission\n")

break.

Explanation:

1. Simulates transmission by four hosts.
2. Detects collisions and perform a random retry.
3. Records transmission events in csma_log.txt.

4Q.

Develop a client-server application using Python's socket library to simulate communication between two hosts.

```
import socket
```

```
s = socket.socket()
```

```
s.bind ("localhost", 5000)
```

```
s.listen (1)
```

```
conn, addr = s.accept()
```

```
data = conn.recv (1024).decode()
```

```
mac = data.split ("", 1)
```

```
print ("Frame:", data)
```

```
print ("MAC Address table:")
```

```
print (mac[0], "→ PORT - 1")
```

```
conn.send ("Frame forwarded".encode()).
```

Conn.close().

1. The client sends a frame containing mac address and data.
2. server received the frame and learns the source mac address.
3. The server displays the MAC table and confirms forwarding.

Q5 Design and implement a python class named starswitch that models the basic functionality.

```
class starswitch:
```

```
def __init__(self):
```

```
    self.ports = []
```

```
def add_port(self, host):
```

```
    self.ports.append(host)
```

```
def remove_port(self, host):
```

```
    for host in self.ports:
```

```
        if host != source:
```

```
            print(f"Frame sent to {host}")
```

```
s = starswitch()
```

```
s.add_port("H1")
```

```
s.add_port("H2")
```

```
s.add_port("H3")
```

1. add_port() and remove_port() connect and disconnect hosts.
2. broadcast() sends the frame to all hosts except the source.
3. The class maintains the list of active connected hosts.